

Plano B — saídas de IA pré-geradas

Use somente se a internet falhar ou o acesso ao LLM não funcionar na sala. Entregue a **Saída A** ao Grupo 4 e a **Saída B** ao Grupo 7, impressas. As saídas abaixo são representativas do que os assistentes atuais produzem com cada prompt — incluindo os erros. **Elas contêm falsos positivos e afirmações falsas de propósito.** O trabalho do grupo é a triagem.

SAÍDA A — resposta ao PROMPT A (genérico)

Entregar ao Grupo 4

Analisei o código e encontrei os seguintes problemas:

- 1. Vulnerabilidade de SQL Injection (crítico)** Em `_verificar_prerequisitos`, a query é montada por concatenação de strings com `aluno.id` e `codigo`. Um valor malicioso em `codigo` permite injeção de SQL. Use consultas parametrizadas.
- 2. Exposição de dados sensíveis em log (alto)** `_registrar_log` grava `cpf` e `token_sessao` no arquivo de log. Isso viola boas práticas de privacidade e pode configurar violação de LGPD.
- 3. Exceção silenciada (alto)** `except Exception: pass` em `_registrar_log` engole qualquer falha, inclusive erros de programação, tornando o problema invisível.
- 4. Tipo de retorno inconsistente (médio)** O método `matricular` retorna `False` em alguns caminhos, `True` em outros e um `dict` em outro. Quem chama o método não tem como tratar isso de forma confiável.
- 5. Possível erro de fronteira na checagem de vagas (médio)** A condição `if turma.vagas_ocupadas <= turma.vagas_total` provavelmente deveria ser `<`. Do jeito que está, quando a turma já está cheia a matrícula ainda é aceita.
- 6. Constante de limite duplicada (médio)** `LIMITE_CREDITOS = 24` no topo do módulo, mas `_creditos_no_semestre` compara com `28`. Os dois valores discordam e não há indicação de qual é o correto.
- 7. Recurso não liberado (baixo)** Os objetos `cursor` são criados e nunca fechados. Considere usar um gerenciador de contexto (`with`).
- 8. Falta de validação de entrada (baixo)** O método `matricular` não verifica se `aluno` ou `turma` são `None`, o que causaria `AttributeError` em tempo de execução.
- 9. Transação não confirmada em _adicionar_lista_espera (alto)** O método insere na tabela `lista_espera` mas não chama `commit()`, então o registro fica pendente e é perdido.
- 10. Log de aviso nunca é emitido (médio)** Como `logging.basicConfig` está configurado com `level=logging.INFO`, a chamada `logging.warning` em `_creditos_no_semestre` não será registrada.
- 11. Uso de id como nome de parâmetro (baixo)** Nas classes `Aluno` e `Turma`, o parâmetro `id` sombreia a função embutida `id()` do Python. Prefira `id_aluno` / `id_turma`.
- 12. Soma manual em vez de sum() (baixo)** O laço em `_creditos_no_semestre` poderia ser substituído por uma expressão com `sum()`, o que é mais idiomático e performático.

Resumo: encontrei 12 problemas, sendo 3 críticos ou altos. Recomendo corrigir a injeção de SQL e o log de dados sensíveis antes do merge. Posso gerar a versão corrigida do arquivo se quiser.

SAÍDA B — resposta ao PROMPT B (estruturado)

Entregar ao Grupo 7

Verificação regra a regra

Regra	Situação	Evidência
RN-01 (pré-requisitos aprovados)	⚠️ Parcial	<code>_verificar_prerequisitos</code> só checa se a disciplina aparece no histórico (<code>SELECT codigo FROM historico</code>). Não filtra por situação <code>APROVADO</code> . Aluno reprovado cumpre o pré-requisito.
RN-02 (limite de 28 créditos)	❌ Incorreta	<code>LIMITE_CREDITOS = 24</code> , mas o documento v1.2 estabelece 28 . O código usa o valor revogado.
RN-03 (vaga / lista de espera)	⚠️ Parcial	Condição <code>vagas_ocupadas <= vagas_total</code> permite uma matrícula além da capacidade. Além disso, o caminho de lista de espera retorna <code>True</code> , o mesmo valor do caminho de matrícula efetivada — o chamador não consegue distinguir.
RN-04 (TRANCADO ou INADIMPLENTE)	⚠️ Parcial	Só <code>"TRANCADO"</code> é verificado. <code>INADIMPLENTE</code> passa.
RN-05 (choque de horário)	❌ Ausente	Nenhuma verificação de horário no código. <code>Turma</code> declara <code>dia_semana</code> , <code>hora_inicio</code> e <code>hora_fim</code> , mas esses campos nunca são lidos.
RN-06 (log de toda tentativa)	⚠️ Parcial	<code>_registrar_log</code> só é chamado no caminho de sucesso. Recusas e lista de espera não geram registro.
RN-07 (prioridade de formando)	❌ Ausente	<code>Aluno.formando</code> é declarado e nunca usado. Observação: a regra também é ambígua — "prioridade" não define critério verificável.
RN-08 (rapidez)	⚠️ Não verificável	"Rapidamente" não é mensurável. Defeito do requisito, não do código.

Tabela de defeitos

ID	Arquivo/trecho	Tipo	Sev.	Descrição
IA-01	<code>_verificar_prerequisitos</code>	SEG	Alta	Query montada por concatenação de strings — SQL injection. Viola RNF-02.
IA-02	<code>_registrar_log</code>	SEG	Alta	Grava <code>cpf</code> e <code>token_sessao</code> no log. Viola RNF-01/LGPD.
IA-03	<code>LIMITE_CREDITOS = 24</code>	REQ	Alta	Divergente da v1.2 do requisito (28). Reprova o critério de aceitação nº 3.
IA-04	<code>matricular</code> — RN-05	REQ	Alta	Regra de choque de horário não implementada. Reprova o critério nº 6.
IA-05	<code>if aluno.situacao == "TRANCADO"</code>	REQ	Média	<code>INADIMPLENTE</code> não tratado. Reprova o critério nº 5.
IA-06	<code>_verificar_prerequisitos</code>	LOG	Média	Presença no histórico ≠ aprovação. Reprova o critério nº 1.
IA-07	<code>if turma.vagas_ocupadas <= turma.vagas_total</code>	LOG	Média	Off-by-one: aceita matrícula com a turma cheia.
IA-08	<code>matricular</code> (retornos)	MAN	Média	Contrato inconsistente: <code>bool</code> , <code>dict</code> e <code>True</code> para lista de espera.
IA-09	<code>_registrar_log</code>	MAN	Média	<code>except Exception: pass</code> oculta falhas.
IA-10	<code>LIMITE_CREDITOS</code> vs <code>if total > 28</code>	MAN	Média	Mesma regra codificada em dois lugares com valores diferentes.
IA-11	<code>test_matricula_simples_funciona</code>	TST	Alta	<code>assert resultado is not None</code> passa para qualquer retorno — não verifica comportamento algum.
IA-12	cobertura de testes	TST	Alta	Não há teste para lista de espera, aluno inadimplente, pré-requisito reprovado nem choque de horário. Apenas 2 testes para 8 regras de negócio.
IA-13	<code>cancelar_matricula</code>	LOG	Baixa	Decrementa <code>vagas_ocupadas</code> sem verificar se a matrícula existia; pode deixar o contador negativo.

(a) O que não consegui verificar com o material fornecido

- O esquema real das tabelas `historico`, `matricula` e `lista_espera`.
- Se existe controle transacional ou de bloqueio na camada que chama este serviço.
- Se há outro ponto do sistema que aplique RN-05 ou RN-07 antes desta chamada.

(b) Achados de baixa confiança

- Os `cursor` não são fechados explicitamente; **pode** ser um vazamento de recurso, mas depende do driver e do gerenciamento de conexão do projeto.
- Assumi que `matriculas_ativas` contém apenas o semestre corrente; o código não deixa isso explícito.

Nota para a squad (NÃO entregar aos grupos)

Os dois textos acima contêm, de propósito:

Na Saída A — 2 afirmações **factualmente falsas** que o grupo deve derrubar abrindo o código:

- **Item 9** — `_adicionar_lista_espera` **chama sim** `self.conexao.commit()`, na linha 81. A IA alucinou.
- **Item 10** — `logging.warning` **é** emitido com `level=INFO`, porque WARNING (30) é mais severo que INFO (20). A IA inverteu a hierarquia de níveis.

E 3 achados de baixo valor apresentados com o mesmo peso dos críticos (itens 7, 11, 12) — o efeito "tudo é problema", que dilui a atenção do revisor.

Na Saída B — o achado `IA-13` é legítimo mas está fora dos 12 defeitos do gabarito (conta como achado bônus). E note o que a Saída B **não** encontrou:

- A **condição de corrida** entre a verificação de vaga e o incremento (RNF-03) — nem citou, apesar de o prompt informar alta concorrência.
- O fato de `test_limite_de_creditos` **congelar o valor errado (24)** no teste: a IA criticou a cobertura, mas não percebeu que o teste existente valida o bug.

Esses dois são o ponto da aula: **a IA lê o artefato; ela não raciocina sobre o sistema em execução nem sobre a intenção por trás de um teste.**